

Introduction → History of Deep Learning:-

- Deep learning has evolved significantly over the decades. It mimics the neural networks of the human brain, enabling computers to autonomously uncover patterns and make informed decisions from vast amounts of unstructured data.

- The origins of deep learning can be traced back to 1943, when Walter Pitts and Warren McCulloch created a computer model based on neural networks. They used a combination of algorithms and mathematics called "threshold logic" to mimic the thought process.

↳ 1950s → The perceptron, a single-layer neural network, was introduced. However, it could only solve linearly separable problems, leading to the development of multi-layer perceptrons (MLPs), which introduced hidden layers and non-linear activation functions.

↳ 1960s → Henry J. Kelley developed the basics of a continuous backpropagation model, which was later

defined by Stuart Dreyfus.

↳ 1970s → The first Artificial Intelligence winter occurred due to unfulfilled promises, limiting research funding. However, Kunihiko Fukushima designed the first convolutional neural networks (CNNs).

↳ 1980s - 1990s → The resurgence of deep learning came with improved training methods, including backpropagation, which allowed neural networks to learn more efficiently.

↳ 2000s - Present → The rise of deep neural networks (DNNs), CNNs, RNNs revolutionized fields like image recognition, natural language processing, and autonomous systems.

- Deep learning continues to evolve, shaping the future of artificial intelligence.

McCulloch Pitts Neuron:-

- The McCulloch Pitts Neuron is one of the earliest computational models of a neuron, developed by Warren McCulloch and Walter Pitts in 1943. It is designed to mimic the basic functioning of biological neurons.

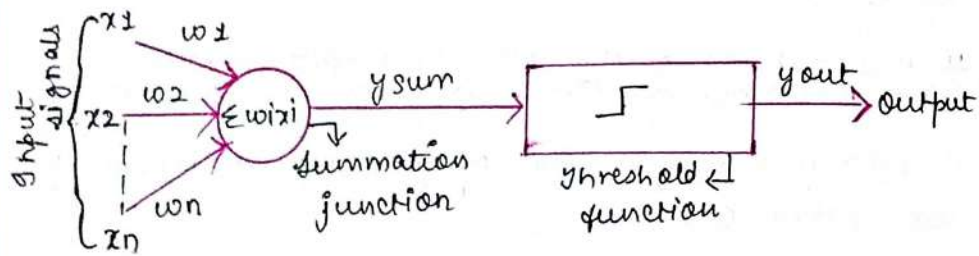
It laid the foundation for artificial neural networks.

→ Key features of the McCulloch-Pitts Neuron:-

- 1) Binary Inputs → The neuron receives inputs that are either 0 or 1.
- 2) Weighted Sum → Each input is assigned a weight, and the weighted neuron computes the weighted sum of all inputs.
- 3) Threshold Activation → If the weighted sum meets or exceeds a pre-defined threshold, the neuron outputs 1; otherwise it outputs 0.
- 4) Logical Operations → The model can represent basic Boolean functions like AND, OR, and NOT making it useful for early AI logic systems.

→ Limitations:-

- 1) It does not support learning, weights and thresholds must be manually set.
- 2) It only works with binary inputs, limiting its complexity.
- 3) It lacks advanced features like backpropagation, which is crucial for modern neural networks to learn and adjust weights effectively.



→ Example :-

John carries an umbrella if it's sunny or if it is raining. There are 4 given situations, I need to decide when John will carry the umbrella. The situations are as follows:-

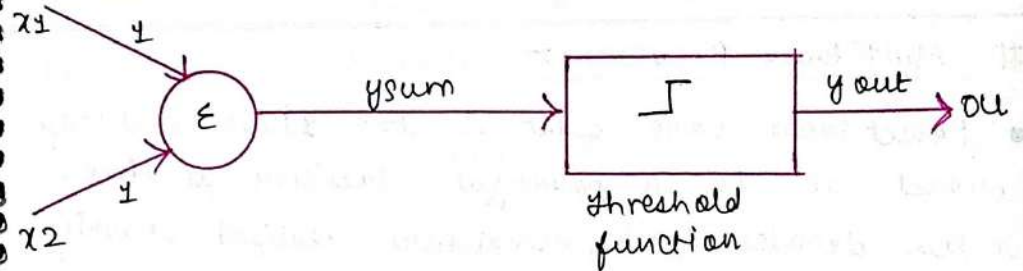
- 1) First scenario:- It is not raining, nor it is sunny.
- 2) Second scenario:- It is not raining, but it is sunny.
- 3) Third scenario:- It is raining, and it is not sunny.
- 4) Fourth scenario:- It is raining as well as it is sunny.

→ To analyze the situations using the Mc-Culloch Pitts Neural model, I can consider the input

signals as follows :-

- x_1 : Is it raining?
- x_2 : Is it sunny?

So, the value of both scenarios can be either 0 or 1. We can use the value of both weights x_1 & x_2 as 1 and a threshold function as 1. So, the neural network model will look like -



→ Truth table for this case -

Situation	x_1	x_2	y_{sum}	y_{out}
1	0	0	0	0
2	0	1	1	1
3	1	0	1	1
4	1	1	2	1

→ So, I can say that -

$$y_{sum} = \sum_{i=1}^n w_i x_i$$

$$y_{out} = f(y_{sum}) = \begin{cases} 1, & x \geq 1 \\ 0, & x < 1 \end{cases}$$

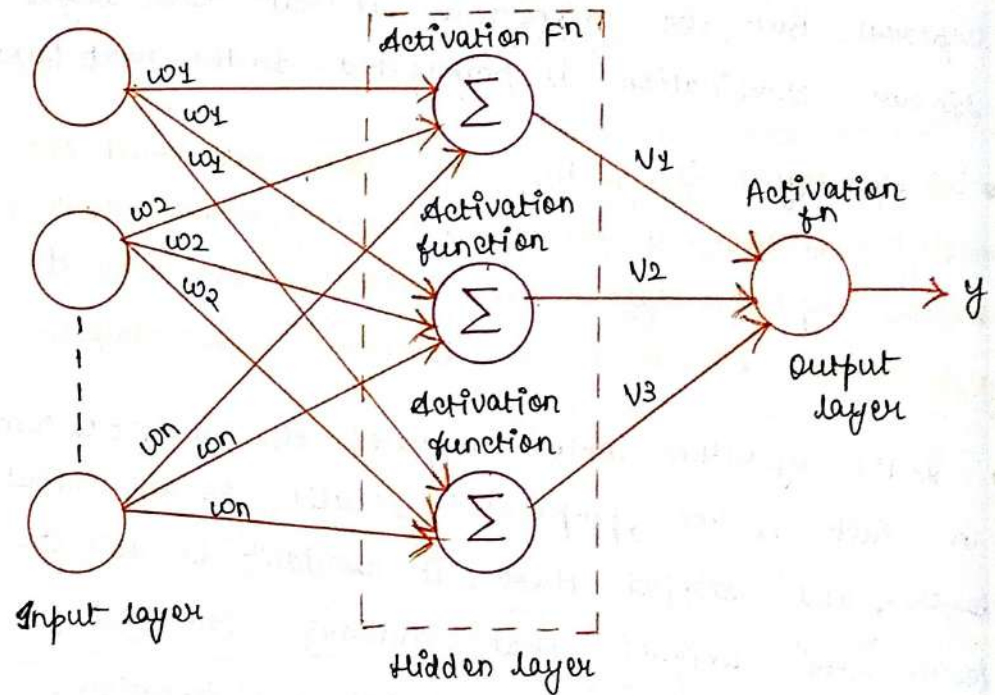
The conclusion that draws out from the truth table is, in situations where the value of y_{out} is 1, John ~~needs~~ ^{does not need} to carry an umbrella. Hence, he will need to carry an umbrella in scenarios 2, 3 and 4.

Multi layer Perceptions:-

- Perceptrons were built in the 1950s. and they proved to be a powerful classifier at time. A few decades later, researchers realized stacking multiple perceptrons could be more powerful. That turned out to be true and multi layer perceptron was born.
- A single layer perceptron works like a neuron in a human brain. It takes multiple inputs and like a neuron emits an electric pulse, a perceptron emits a binary pulse which is treated as a response.
- The neuron-like behaviour of perceptrons and an MLP being a network of perceptrons perhaps led to the term neural network come forth in

the early days.

- Since, its creation, neural networks have come a long way. Tremendous advancements in various architecture such as convolutional neural networks, recurrent neural networks, etc have been made.
- Multi layer perceptrons are complex non-linear models. MLP is a collection of simple regression model placed on every node in each layer.



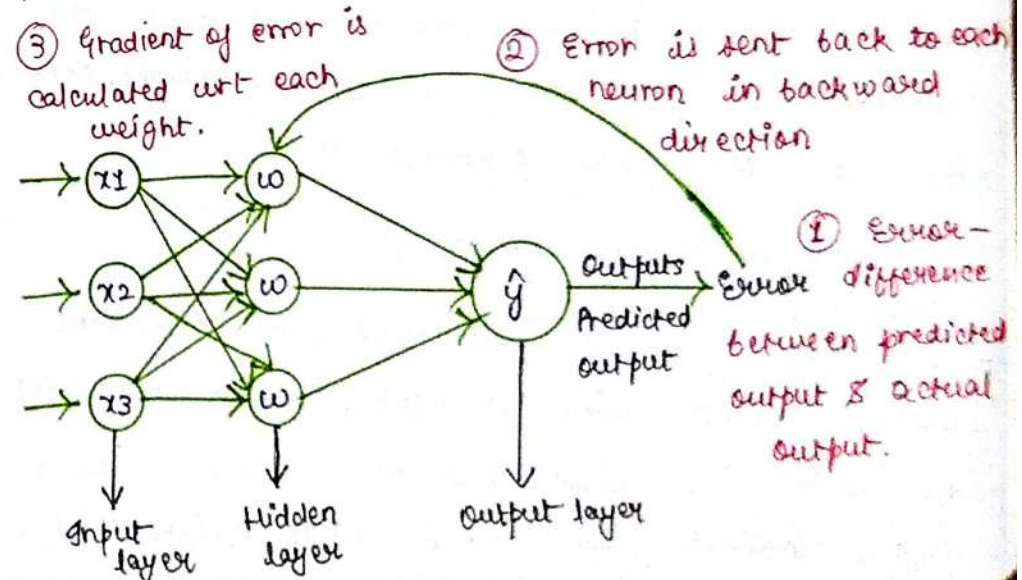
- A multi layer perceptron has input and output layers, and one or more hidden layers with many neurons stacked together. And while in the percep

then the neuron must have an activation fn that imposes a threshold, like RELU, or sigmoid, neurons in a MLP can use any arbitrary activation function.

- MLP falls under the category of feedforward algorithms, because inputs are combined with the initial weights in a weighted sum and subjected to the activation function, just like in the perceptron. But the difference is that each layer's linear combination is propagated to the next layer.
- Each layer is feeding the next one with the result of their computation, their internal representation of the data. This goes all the way through the hidden layers to the output layer.
- If the algorithm only computed the weighted sums in each neuron, propagated results to the output layer, and stopped there, it wouldn't be able to learn the weights that minimize the cost fn. If the algorithm only computed one iteration, there would be no actual learning, and this is where Backpropagation comes into play.

Backpropagation :-

- Backpropagation is the learning mechanism that allows the Multi-layer Perceptron to iteratively adjust the weights in the network, with the goal of minimizing the cost function.
- There is one hard requirement for backpropagation to work properly. The function that combines inputs and weights in a neuron, for instance the weighted sum, and the threshold function, for instance RELU, must be differentiable.
- These functions must have a bounded derivative, because Gradient descent is typically the optimization function used in MLP. It involves two main passes $\rightarrow$ forward and backward pass.



1) Forward pass → Input is fed into the input layer and these inputs combined with their respective weights are passed to hidden layers. Before applying an activation function, a bias is added to the inputs.

→ Each hidden layer applies, RELU activation function which returns the input if it's positive and zero otherwise. This adds non-linearity allowing the model to learn complex patterns of data.

→ Finally, the output is passed to the output layer.

2) Backward pass → In Backward pass, error, which is the difference between the actual and the predicted value, is propagated back through the network to adjust the weights and biases. Method used in MSE, the formula is —

$$MSE \rightarrow (\text{Predicted} - \text{Actual})^2$$

→ Once the error is calculated, the network adjusts weights using gradients, which are computed with the chain rule of calculus. These gradients indicate how much each weight and

bias should be adjusted in order to minimize the error in the next iteration.

• The backward pass continues layer by layer ensuring that the network learns and improves its performance.

→ Advantages :- 1) Easy implementation.

2) Efficiency

3) Scalability

4) Generalization

→ Disadvantages :- 1) Vanishing gradient problem.

2) Overfitting

3) Exploding gradients

→ Difference between FFN and BP

Feed-Forward Neural Network

1) A type of neural network where data flows in one direction from inputs to output.

2) Unidirectional, from the input to the output.

Backpropagation

1) A training algorithm used to adjust weights and biases to minimize loss.

2) Bidirectional, with forward pass for predictions & backward pass for error correction.

3) used to make predictions on the input data.

4) involves processing inputs through layers to generate an output.

5) Simplex.

6) Defines the structure & prediction mechanism of network.

7) Does not involve learning.

8) Does not handle errors directly; outputs are generated without feedback.

9) A single pass of data through the network.

10) Involves input, hidden & output layers.

3) used to train the network by optimizing the weights to reduce prediction errors.

4) Involves calculating the error, finding gradients, and updating weights iteratively.

5) Complex.

6) Applied during training to improve the network's accuracy.

7) Essential for learning.

8) Directly handles errors by propagating them backward to update the weights.

9) Requires multiple iterations for effective training.

10) Interacts with all the layers.

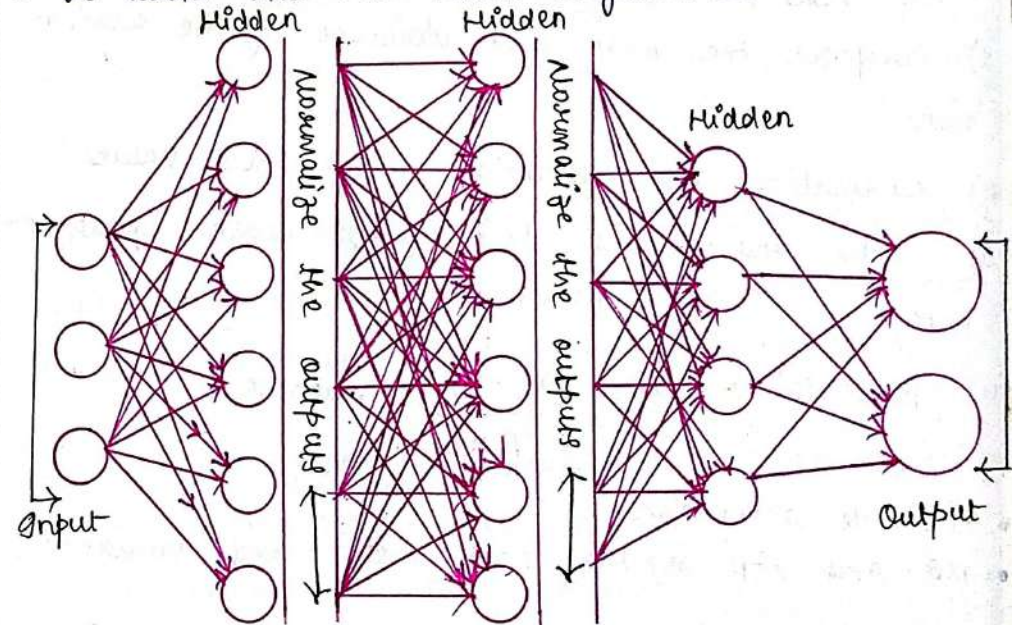
Batch Normalization:-

- Batch Normalization is an algorithm which makes the training of deep neural networks faster and more stable.

- It consists of normalizing activation vectors from hidden layers using the mean and variance of the current batch.

- This normalization technique is applied right before or after the activation. The inputs of each layer are normalized before output generation of that layer.

- It looks like the below diagram —



→ why use BN:-

1) Increases efficiency of Neural Network:- It helps in faster convergence by normalizing input at each layer.

2) Reduce internal covariate shift:- covariate shift happens when distribution of input changes between training and live environments. The output distribution remains same.

→ Internal covariate shift refers to the change in the distribution of network activations due to change in network parameters during training.

→ Working:- For each mini-batch, BN:-

1) Computes the mean and variance of the activations.

2) Normalizes the activations using these values.

3) Scales and shifts them using learned parameters.

4) Formula → $\hat{x}_i = \frac{x_i - \mu_\beta}{\sqrt{\sigma^2_\beta + \epsilon}}$, where,

• x_i is an activation

• μ_β and σ^2_β are the both mean and variance,

• ϵ is a small constant to avoid division by 0.

Decomposition - PCA & SVD:-

• Decomposition is an important technique used to break complex problems into smaller, more manageable components. This helps in improving efficiency, interpretability, and performance in various tasks.

• Here are some common types of decomposition in deep learning -

1) Feature decomposition → Splitting data into meaningful features, which can help models learn better representations.

2) Model decomposition → ^{div}splitting a deep learning into smaller sub-models or layers that specialize in specific tasks.

3) Matrix decomposition → Techniques like singular value decomposition (SVD) and Principal component analysis that help in dimensionality reduction.

4) Task decomposition → Breaking a complex task into smaller sub-tasks that can be solved individually before combining results.

1) PCA → Principal Component Analysis:-

- PCA is a technique often used to reduce the dimensionality of large datasets while preserving as much variance as possible.
- It transforms data into a new coordinate system where the most significant features are retained.
- In simpler terms, it's about finding the underlying structure in the data by identifying the "directions" along which the data varies the most. These directions are called principal components.
- PCA is useful for speeding up model training, removing noise, and enhancing generalization.

2) SVD → Singular Value Decomposition:-

- SVD decomposes a matrix into three smaller matrices, helping identify essential features and relationships in data.

→ U : a matrix that represents the left singular vectors.

→ Σ : a diagonal matrix with singular values.

→ V^T : a matrix that represents the right singular vectors.

- In mathematical terms, if we have matrix A , SVD decomposes it as -

$$A = U \Sigma V^T$$

- SVD is widely used for compressing neural network weights, reducing computation costs, and handling large-scale data efficiently.

→ Relationship between SVD & PCA -

1) Mathematical connection → PCA and SVD both decompose a data matrix to identify patterns and reduce dimensionality. PCA finds principal components by computing the eigen vectors of the covariance matrix, while SVD directly decomposes the data matrix into singular vectors and singular values.

2) Process linkage → When you apply PCA, you first center the data and compute the covariance matrix. Then, eigen decomposition is performed, alternatively, you can directly apply SVD on the mean-centered data matrix, and the singular vectors will correspond to the principal components.

3) Computational efficiency → SVD is often more stable and numerically efficient compared to eigen decomposition in PCA, especially for large datasets. DL frameworks frequently use SVD for feature extraction, compression and regularization.

GPU :-

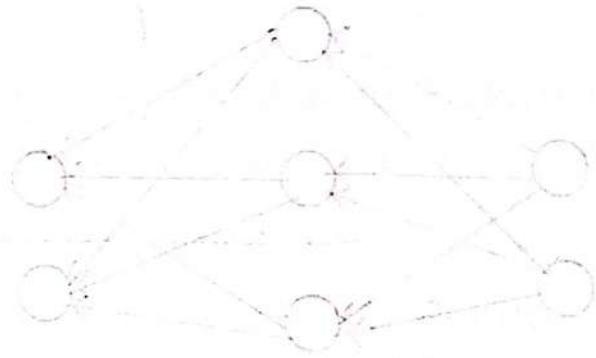
- Training a neural network is very computationally intensive, and because these computations can very easily be parallelized, they call for a new approach to hardware.
- Graphical Processing units (GPU), originally designed for the gaming industry, having a large number of processing cores and very large on-board RAM,
- GPUs are increasingly used for deep learning applications and can dramatically accelerate neural network training.
- GPUs matter in deep learning because —
 - 1) High speed computations
 - 2) Parallel processing
 - 3) Optimized for deep learning

Popular GPU technologies in DL are —

- 1) NVIDIA CUDA.
- 2) Google Coral.
- 3) GPUs vs TPUs.

Deep learning models use GPU like —

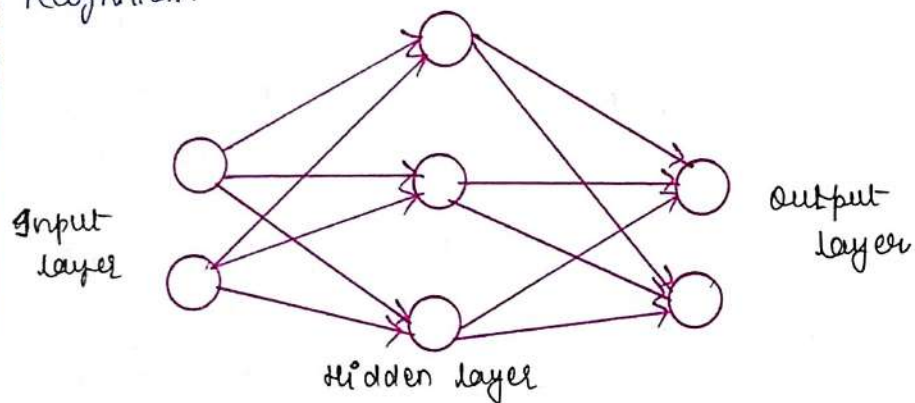
- 1) Training :- GPUs reduce the time required to train models, making it feasible to experiment with complex architectures.
- 2) Inference :- even after training, GPUs help the real-time processing critical for applications like autonomous vehicles and speech recognition.
- 3) Model optimization :- techniques like quantization, and pruning run efficiently on GPUs.



UNIT-2

Feed Forward Neural Network :-

- A feedforward neural network is a type of neural network where the data flows only in one direction from input layer to output layer, without any feedback loops.
- Each layer consists of nodes that apply a non-linear transformation to the input data.
- The output of each layer is used as the input to the next layer. Feedforward neural networks are commonly used for image classification, speech recognition, and other tasks that require pattern recognition.



- The structure of a feedforward neural network:-

1) Input layer → consists of neurons that receives

the input data. Each neuron in the input layer represents a feature of the input data.

2) Hidden layer → these learn the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.

3) Output layer → It provides a final output of the network.

- Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.

- The steps to train a Feedforward neural network are as follows -

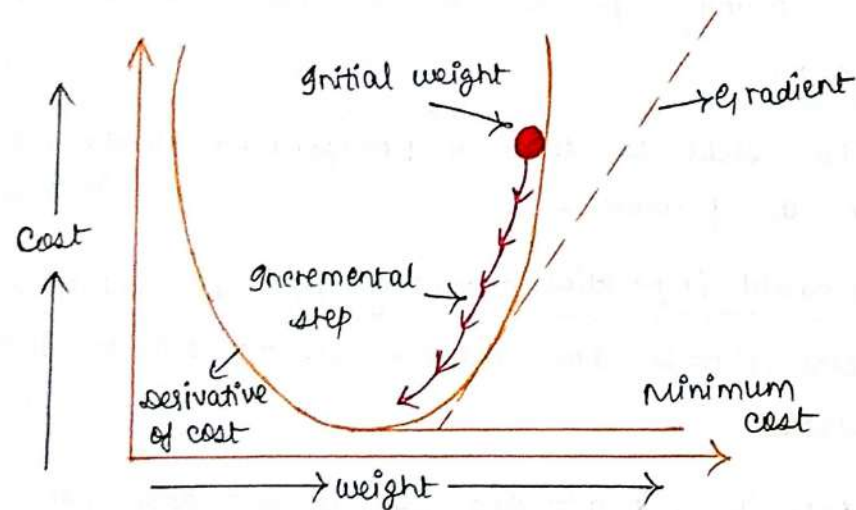
4) Forward Propagation :- Using FP, the input data passes through the network and the output is calculated.

2) Loss Calculation :- The loss is calculated using a loss function such as MSE for regression tasks.

3) Backpropagation :- In BP, the error is propagated back through the network to update the weights.

Deep learning $\rightarrow$ Gradient descent algorithm:-

- Gradient descent algorithm in deep learning is used to optimize the weights and biases of the neural networks.
- The goal is to minimize the loss function, which measures the difference between the network's predictions and the actual labels.
- Gradient descent iteratively adjusts the weights & biases to reduce the loss, allowing the network to learn from the data.



- It first initializes the model's parameters randomly or using the pre-trained model.
- Calculate the loss b/w the predicted output &

the actual label using a loss function.

- Compute the gradients of the loss with respect to the model's parameters using backpropagation.
- Update the model's parameters using the gradients and a learning rate.
- Repeat steps 2-5 for multiple iterations or until convergence.

$\rightarrow$ Reasons to use in DL:-

- 1) Optimization algorithm, that can efficiently minimize the loss functions, even for complex models with millions of parameters.
- 2) DL models often have non-convex loss functions, which can have multiple local minima. GD can handle non-convex optimization problems, making it suitable for deep learning.
- 3) GD can handle large datasets by processing them in batches, making it scalable for big data applications.
- 4) GD can be used with various loss functions, activation functions, and optimization techniques, making it a versatile algorithm for DL.

→ Variants of GD:-

1) Momentum-Based GD:- adds a 'velocity' term to help smooth out updates and avoid getting stuck in local minima.

$$w_t = w_{t-1} + v_t$$

→ think of rolling a ball down a hill - it gains speed and doesn't stop at every small bump. This helps speed up learning and avoid getting stuck.

2) Nesterov Accelerated Gradient (NAG):- similar to Momentum GD but anticipates future updates, making corrections more precise. It looks ahead before updating weights, improving convergence.

→ Instead of blindly following gravity, this method looks ahead before making a move, making it more precise and efficient.

3) Stochastic Gradient Descent (SGD):- updates the weights after each sample instead of the entire dataset.

$$w = w - \alpha \cdot \frac{\partial L}{\partial w}$$

→ Instead of looking at the entire hill, it takes tiny steps based on small parts, making learning faster but a little shaky.

4) AdaGrad:- Adaptive Gradient algorithm; adjusts the learning rate dynamically for each parameter based on historical gradients.

→ adjusts its speed based on past movements. If it's been moving too fast, it slows down; if it's stuck, it speeds up.

5) Adam:- Adaptive Moment Estimation is like an upgraded version of GD that combines the best features of Momentum-based GD and AdaGrad to make learning more efficient. It uses two tricks:

1) Velocity (Momentum) to speed up learning.

2) Scaling factor to ensure stable updates.

→ Imagine you're learning to ride a bicycle, and if you keep falling in certain directions, you start adjusting your movements based on past mistakes.

→ Adam does the same - it looks at past weights updates and smartly adjusts how fast it learns.

Autoencoders :-

- An autoencoder is a type of neural network that learns to compress data into a smaller, more meaningful representation, then reconstruct it back. It consists of two parts -

1) Encoder :- Compress input into fewer features.

2) Decoder :- Rebuilds the original data from the compressed version.

- Autoencoders are used in data compression, noise removal, and feature learning.

→ Regularization in Autoencoders :-

- Regularization techniques prevent overfitting, meaning the model doesn't memorize the data but instead learns useful features. Some methods are:

1) Adding noise to inputs (forcing the network to ignore noise and focus on important patterns).

2) Sparse activation of neurons (ensuring the network only focuses on key features).

3) Penalty terms on weights (keeping model complexity low).

- Regularization is used to encourage the network

to learn a sparse representation of the input data.

→ Key Techniques :-

1) Sparsity regularization :- Forces the network to activate only a few neurons per input, making it focus on essential features rather than redundant information.

2) Noise addition :- Denoising autoencoders; introduce random noise to inputs but expect the model to reconstruct clean data.

3) Contractive Regularization :- Limits the sensitivity of hidden layers to small changes in input. Adds a penalty to the loss functions so that slight variations don't drastically change the reconstructed output.

4) Dropout :- Temporarily ignores random neurons during training to make learning more distributed & avoid dependence on specific connections.

- Autoencoders prevent overfitting, encourage feature learning and improve robustness.

5) Variational Autoencoders (VAE) :- VAE don't just

compress data - they generate new variations, of similar data. They assume data is probabilistic, making them powerful for generating realistic images.

→ Key components of VAE -

1) Encoder → Instead of storing compressed data in fixed numbers, it stores them as probability distributions.

2) Latent space representation → This is where the model learns the overall structure of data and generates new variations from it.

3) Decoder → It reconstructs the output based on the sampled values from the latent space, and allows the creation of new examples.

→ Example -

Imagine you're learning how to draw a cat, of different types of cats. A basic autoencoder would compress and reconstruct images of existing cats. But a VAE doesn't just memorize images - it learns the "essence" of cat features and then can create entirely new cat images that don't exist in the dataset.

→ Advantages:-

- 1) Data generation
- 2) Smooth feature learning
- 3) Better compression

• It is widely used for tasks like creating realistic images, synthetic data generation and even AI-based art.

△ Sparse autoencoders:-

• A sparse auto-encoder is a type of neural network that is trained to reconstruct its input data.

• Sparse autoencoders are useful for -

1) Dimensionality reduction:- Reduce the number of features in high-dimensional data while retaining most of the information.

2) Feature extraction:- Extracts relevant features from the data.

3) Anomaly detection:- Detects outliers and anomalies in the data.

→ Example:- Imagine you're analyzing pictures of cats and dogs. Instead of memorizing every pixel,

a sparse autoencoder focuses on the essential features - like ears, tails, and fur patterns - so it can recognize them even in new images.

- To enforce sparsity, we limit the number of activated neurons in the hidden layers using L1-regularization or KL divergence.
- It prevents unnecessary complexity in learning. Helps discover underlying patterns efficiently. It also works well for large datasets where raw features are overwhelming.
- Sparse autoencoders are widely used in image recognition, anomaly detection and data compression.

Data augmentation:-

Data augmentation is a technique used to artificially increase the size of a training dataset by applying transformations to the existing data.

These transformations can include -

- Rotation
- Flipping
- Scaling

iv) Translation

v) Color jittering

vi) Noise injection

- By augmenting the dataset, you can help the model generalize better to new, unseen data, and reducing overfitting, improves its overall performance.

- Data augmentation is important for several reasons:-

- Limited data
- Overfitting
- Robustness

Techniques for dataset augmentation -

- Random cropping → Involves randomly cropping a portion of an image horizontally or vertically to create a new data sample.
- Random flipping → Involves randomly flipping an image to create a new data sample.
- Color Jittering → This involves randomly changing the brightness, contrast and saturation of an image to create a new data sample.

iv) Gaussian Blur:- This involves applying a gaussian blur to an image to create a new data sample.

→ For text:

i) Synonym replacement:- Swap words with similar meanings.

ii) Sentence paraphrasing:- Rewrites sentences while keeping intent.

iii) Adding noise:- Includes small typos or punctuation changes for robustness.

→ For Audio:

i) Speed & Pitch changes:- Modifies tempo or tone without altering meaning.

ii) Background noise addition:- Adds slight disturbances for real-world effectiveness.

• Advantages:- 1) Reduces overfitting.

2) Improves model generalization.

3) Compensates for limited data.

• Data augmentation is widely used for image recognition, speech processing and NLP.

UNIT-3

Convolutional Neural Networks:-

• A CNN is a type of deep learning model designed to process data with a grid-like structure, such as images. Unlike traditional neural networks, CNNs use layers that specialize in detecting patterns, edges, texture, and objects within an image.

• It works by -

1) Convolutional layers → These layers apply filters to extract features like edges, shapes and textures from an image, and scanning the image in both horizontal and vertical directions.

2) Activation function → Introduces non-linearity to the model, allowing it to learn complex patterns. Common activation functions include Rectifier linear unit (ReLU).

→ ReLU = After every convolutional layer operation, ReLU operation has been used to rectify non-linear activation function.

ReLU is an element wise operation that means it applied per pixel and replace all negative pixel values

in the feature map by 0.

Convolutional is a linear operation or non-linear is produced by using non-linear activation function like ReLU.

0	2	-1	3	-2
-4	0	-3	2	-3
-0	4	1	2	-4
-1	3	4	-3	-9
9	-2	1	1	-3
1	-4	0	5	-6

feature map

ReLU

0	2	0	3	0
0	0	0	2	0
0	4	1	2	0
0	3	4	0	0
9	0	1	1	0
1	0	0	5	0

3) Pooling layer → These layers reduce the spatial dimensions of the image, helping to downsample the image. It reduces the computational complexity.

Since the number of hidden layers required to learn the complex relation present in the image could be large. We apply pooling operation to reduce the input features representation, thereby reducing the computational power required by the network.

Special pooling can be different types -

1) Max pooling

2) Average pooling

3) Sum pooling

1	4	4	5
0	1	3	2
8	2	3	5
4	1	2	2

4x4

2x2 filter
stride 2

P=0

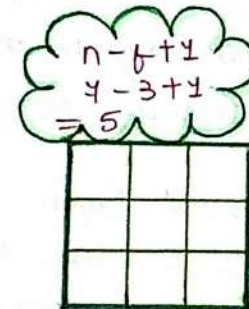
max pooling operation

4	4
8	5

4) Padding :- Padding refers to adding extra pixels around the edges of an input image before applying convolutional operations. It helps maintain the spatial dimensions of the image and can improve feature extraction.

0	1	0	1	1	0	1
0	0	1	1	1	0	1
0	1	1	1	0	0	0
0	1	0	0	1	0	0
0	0	1	0	0	1	0
0	1	0	0	1	1	0
0	1	0	1	0	0	1

7x7

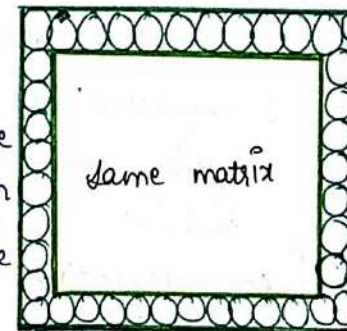


3x3

∴ formula
 $n + f + 1 \neq$
 $n + 2(P) - f + 1$

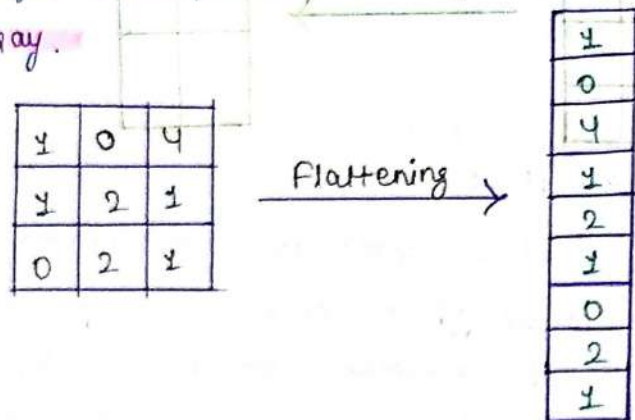
Padding →

applying the padding of n on the above matrix →



$n + 2(P) - f + 1$
 $4 + 2(1) - 3 + 1$
 $4 + 2 - 3 + 1$
 $9 - 3 + 1$
 $6 + 1$
 $= 4$

5) Flattening:- It flattens or reorganizes the entire feature map into a single, long vector or 1D-array.

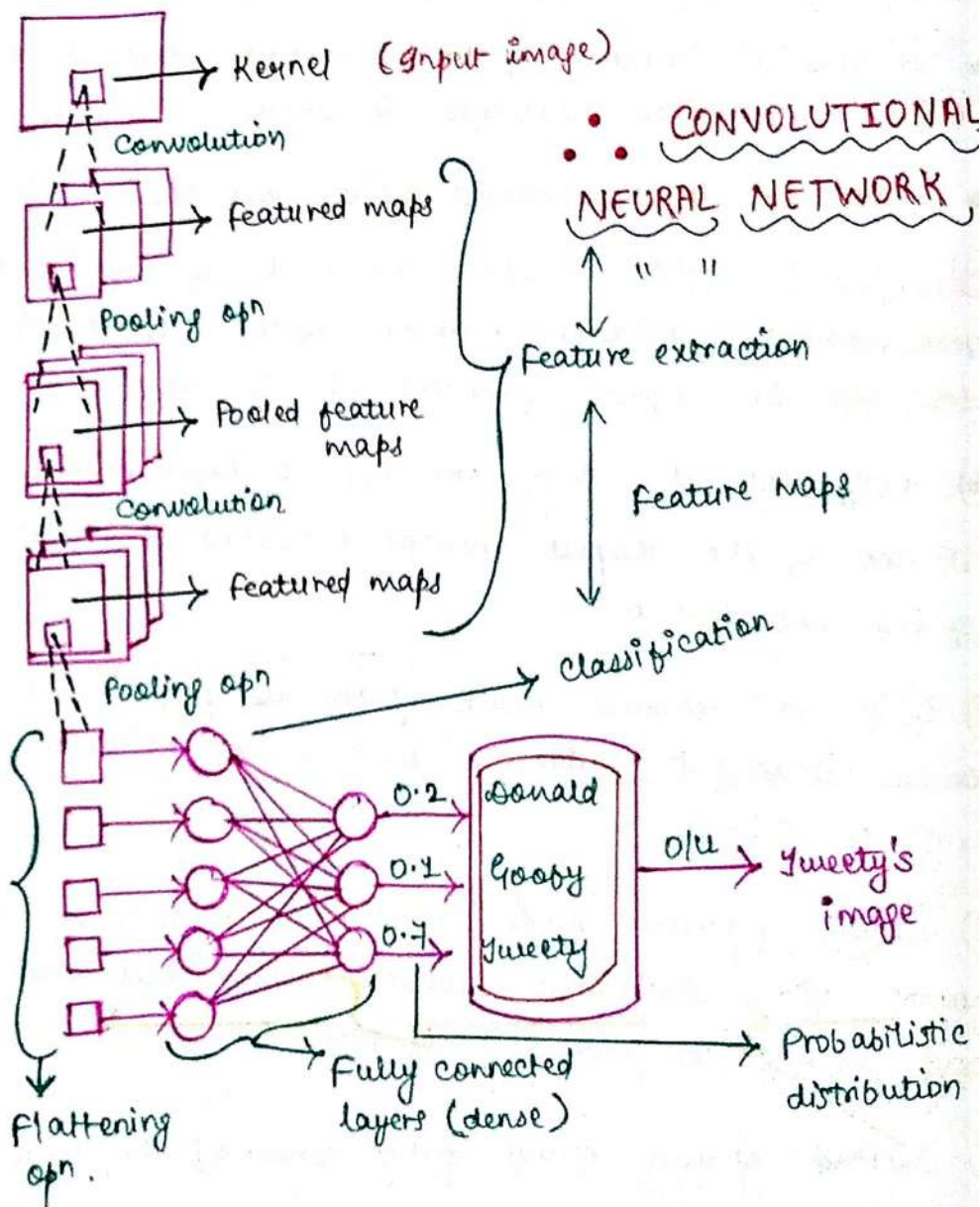


By flattening the feature maps into a vector, the network can integrate all the spatially distributed features extracted for tasks such as classification.

Fully connected layers (dense layers) are designed to operate on 1-D data, hence, flattening is a necessary step to transition from the multidimensional tensors produced by convolutional layers to the format required for dense layers.

6) Fully connected layers:- Consists of multiple dense layers, where each neuron is connected to every neuron in the previous layer. While convolutional layers are good at detecting features in input data, dense layers are essential for integrating these features into predictions.

→ Without dense layers, CNNs would not be able to perform the high-level tasks that are often required, such as classifying images, detecting objects, or making predictions based on visual inputs.



Short notes on —

1) Google Net:-

- GoogleNet is a deep neural network architecture that was designed by Google researchers in 2014.
- It was the winner of the ImageNet large scale visual recognition challenge in 2014.
- Google Net is characterized by its use of —

a) Inception modules:- These are blocks of layers that use multiple parallel branches with different filter size to capture features at dif scales.

b) Deep network:- Google Net has 22 layers, making it one of the deepest neural networks at the time of its introduction.

c) Google Net achieved state-of-the-art performance on the ImageNet dataset, with a top-5 error rate of 6.67%.

d) Unlike previous model, Google Net reduces the number of calculations while keeping high accuracy, making it fast and efficient.

e) Instead of using many fully connected layers, it

averages the feature maps, reducing the chances of overfitting.

- GoogleNet is used in applications like object detection, medical imaging and autonomous vehicles.

• Advantages → Better accuracy, computational efficiency, handles complex patterns, reduced overfitting and deeper but optimized.

• Imagine looking at an image —

- 1) Small (1×1) filters focus on fine details like edges.
- 2) Medium (3×3) filters focus on capturing textures & patterns.
- 3) Large (5×5) filters detect bigger shapes and structures.

By combining all these, GoogleNet learns more efficiently without increasing computational cost too much.

2) Sigmoid Neurons:-

- Sigmoid neurons are a type of artificial neuron that uses the sigmoid activation function.

• The sigmoid function maps the input to a value between 0 and 1, making it suitable for binary classification problems.

• $\sigma(x) = 1 / (1 + \exp(-x))$, where,

↳ x is the weighted sum of inputs plus bias.

↳ e is the Euler's number (2.71).

↳ The function transforms the input into a range between 0 and 1.

• It handles non-linearity, and makes training with gradient descent easy.

• It helps interpret results as probabilities.

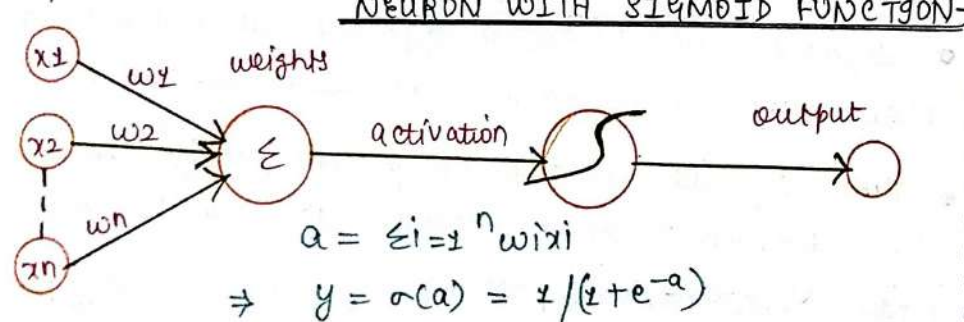
• Limitations:- Vanishing gradient problem, output not centered around zero and expensive computation.

• Usage:- 1) Binary classification problems, for eg. spam and non-spam detection.

2) Probabilistic predictions in ML models.

3) Logistic regression for simple classification tasks.

inputs



3) LeNet:-

• LeNet is a convolutional neural network architecture that was designed by Yann Lecun and his colleagues in the 1990s.

• It was one of the first CNNs to be applied to image recognition tasks.

• LeNet consists of -

1) Convolutional layers → These layers use filters to extract features from the input image.

2) Pooling layers → These layers are used for classification downsampling the feature maps to reduce spatial dimensions.

3) Fully connected layers → These layers are used for classification.

• Before deep learning became mainstream, recognizing handwritten digits was a challenging problem. LeNet solved this by introducing CNNs, making computers automatically extract patterns from images instead of relying on handcrafted features.

• Key features → 1) uses convolution.

2) Pooling layers reduce complexity.

3) Fully connected layers for final classification.

4) Efficient for small images.

4) AlexNet :-

- AlexNet was developed by Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton in 2012.

- It won the ImageNet large scale visual recognition challenge and marked the beginning of the deep learning's dominance in computer vision.

- Before ALEXNET, image classification models were shallow, traditional machine learning techniques & struggled with complex datasets.

- AlexNet introduced deep architecture, ReLU fⁿ, dropout regularization and GPU acceleration.

- These advancements allowed AlexNet to dramatically outperform all previous models, proving that deep learning could recognize images with high accuracy.

- Architecture of AlexNet includes -

1) Input layer :- Takes a 224×224 RGB image as input.

2) Convolutional layer :- Applies 96 filters of size

11×11 with ReLU activation, extracting edges and textures.

3) Max Pooling :- Reduces features size, making computation more efficient.

4) Convolution layers 2, 3, 4, 5 :- Extract deeper features using smaller filter sizes (5×5 , 3×3).

5) Fully connected layers :- Flatten the feature maps and pass them through dense layers to classify the image.

6) Softmax layer :- Produces probabilities for 1000 dif classes (e.g, dog, cat, airplane).

- AlexNet is used in self driving cars, medical diagnosis, industrial automation and image recognition.

- AlexNet proved deep learning's power, and today, advanced networks build upon its foundation for even greater performance.

5) ResNet :-

- Residual network is a groundbreaking CNN architecture introduced by Microsoft researchers Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun in 2015.

- It won the ImageNet classification competition, proving that extremely deep neural networks could be trained effectively without performance degradation.

• The key features of Resnet are —

- 1) Residual learning :- Instead of directly learning a mapping, Resnet learns the residuals between input and output, making training easier.
- 2) Skip connections :- Resnet introduces shortcut connections that bypass layers, allowing the network to retain information from earlier layers.
- 3) Deep architecture :- Resnet can go hundreds of layers deep (eg - Resnet-50, 101, 152) unlike traditional CNNs, which struggle to scale efficiently.
- 4) Batch normalization :- It stabilizes training and accelerate convergence.

• Each residual block follows this pattern:-

- 1) Input (x) → Convolutional layers + Batch normalization.
- 2) Skip connection → Adds the original input (x) back

to the processed output.

- 3) Final activation :- ReLU to introduce non-linearity.

Mathematically, the final output is written as —

$$H(x) = F(x) + x$$

- Resnet solves the vanishing gradient issue, with the help of skip connections which preserve gradient flow.
- Resnet performs exceptionally well on image recognition tasks.
- Useful for applications like object detection and medical imaging.

6) ZF-Net :-

- ZF-Net (Zeiler & Fergus Network) is a deep learning architecture that builds upon AlexNet but introduces improvements to enhance feature visualization and network performance. It was developed by Matthew Zeiler and Rob Fergus in 2013.

• Key features of ZF-Net are —

- 1) Inspired by Alexnet :- It follows a similar convolutional structure but modifies the filter size in

early layers.

2) Improved feature visualization:- It enables better understanding of what each layer learns by using deconvolutional networks.

3) Modifications in Kernel size:- The first convoluted layer uses a 7×7 filter instead of AlexNet's 11×11 , allowing for better fine-grained feature extraction.

4) Enhanced training stability:- Adjusted hyperparameters provide better convergence during training.

• Architecture:- Consists of five convolutional layers followed by three fully connected layers -

1) Conv 1: 96 filters of 7×7 , stride = 2, captures low-level features.

2) Conv 2: 256 filters of 5×5 , extracts complex textures.

3) Conv 3, 4, and 5: Smaller filters (3×3) to refine higher-level patterns.

4) Fully connected layers: Process extracted features and classify images.

• Advantages and impacts of ZFNet are -

→ Improved accuracy over AlexNet in ImageNet classification.

→ Helped visualize how CNNs process images.

→ Provided insights for future architectures like VGG-Net and ResNet.

7) VGG-Net:-

• It is a deep convolutional neural network (CNN) architecture developed by the Visual Geometry Group at the University of Oxford.

• It was introduced by Simonyan and Zisserman in 2014 and achieved state-of-the-art performance in the ImageNet competition.

• Key features of VGG-Net -

1) Deep architecture:- Unlike earlier models, VGG-Net uses very deep networks (eg, VGG-16, 19) with up to 19 layers.

2) Small kernel size:- Uses 3×3 convolutional filters, instead of larger ones, allowing better feature extraction.

3) Uniform design:- Every convolutional layer follows the

same filters size and padding, making the structure clean and efficient.

4) Fully connected layers:- Includes 2 hidden layers and a softmax output layer for classification.

5) Pretrained model:- VGG-net is widely used for transfer learning in image recognition tasks.

• Architecture:- VGG-net primarily consists of multiple convolutional layers stacked together, followed by fully connected layers.

1) Conv layers:- uses 3×3 filters with ReLU activation, allowing deeper feature extraction.

2) Max-Pooling layers:- Reduces spatial dimensions and prevent overfitting.

3) Fully-connected layers:- Process extracted features for final classification.

4) Softmax layer:- Outputs probability distributions over classes.

• Impact and limitations -

1) Improved accuracy in image classification tasks.

2) Strong feature extraction for transfer learning in

medical imaging, object detection and face recognition.

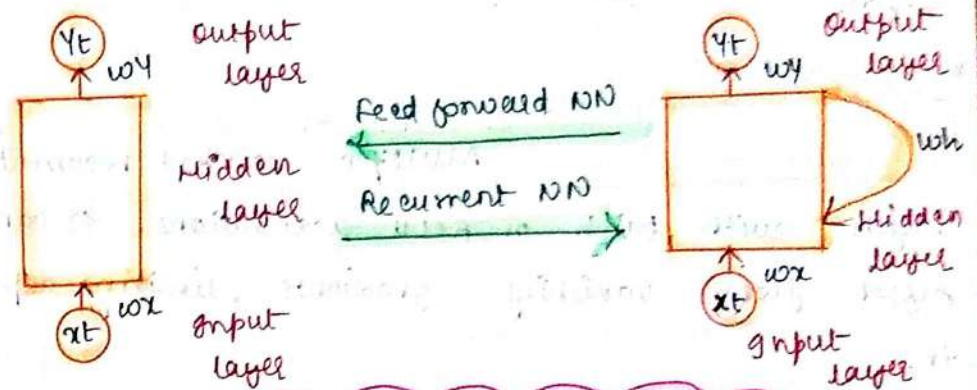
3) High computational cost due to deep architecture.

4) Requires more memory than ResNet or MobileNet.

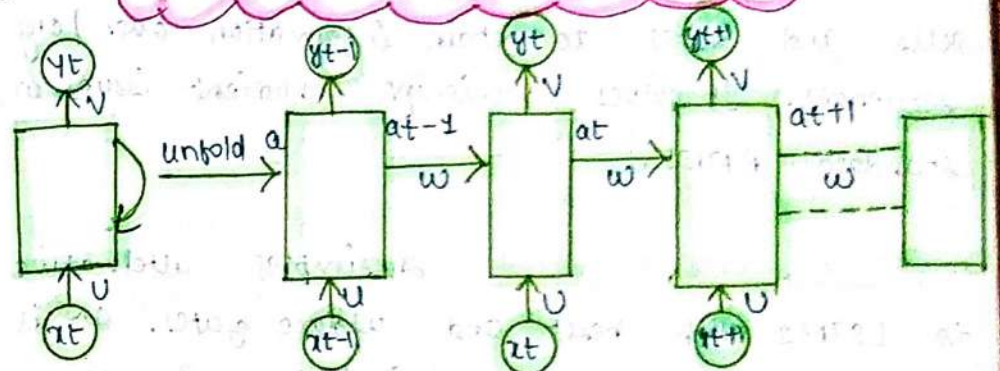
UNIT-4

Deep Recurrent Neural Networks:-

- Recurrent neural networks were introduced to address the limitations of traditional neural networks like feedforward (FNN) when it comes to processing sequential data.
- FNN takes input and processes each input independently through a number of hidden layers without considering the order and context of other inputs.
- Due to which it's unable to handle sequential data effectively and capture the dependencies between inputs.
- As a result, FNN are not well-suited for tasks such as language modelling, machine translation, speech recognition, time series analysis and many other.
- RNN overcomes these limitations by introducing a recurrent connection that allows information to flow from one time step to the next.



ARCHITECTURE OF DEEP RNN :-



- This recurrent network allows/enables to maintain in internal memory, where the output of each step is fed back as an input to the next step, allowing the network to capture the information from previous steps and utilize it in the current step, enabling the model to learn the temporal dependencies & handle input of variable

length.

• Architecture:-

1) Vanilla Deep RNN:- Multiple stacked recurrent layers with tanh or ReLU activations. It can suffer from vanishing gradients, limiting depth.

2) Long short-term memory:- Uses memory cells and gates to retain information over long sequences. It solves vanishing gradient issues in standard RNNs.

3) Gated Recurrent unit:- A simpler alternative to LSTMs with reset and update gates. It is computationally efficient while preserving long-term dependencies.

4) Bidirectional RNNs:- Processes sequences in both directions to capture more context. It is used in speech processing and text analytics.

5) Echo state networks:- Uses a reservoir computing approach, where weights in the hidden layer

remain fixed. It is suitable for chaotic time-series predictions.

• Applications:- speech recognition, medical diagnosis, chatbots & conversational AI, machine translation.

LSTM:-

• LSTM (Long short term memory) is a type of RNN designed to overcome the limitations of traditional RNNs, particularly the vanishing gradient problem.

• It enables efficient learning of long-range dependencies, making it highly useful for sequential data tasks like speech recognition, text processing, and time-series prediction.

• Key features of LSTM -

1) Memory cells:- Retains information over long sequences, preventing data loss from earlier time steps.

2) Gated mechanism:- Utilizes three gates (input gate, forget gate, output gate) to control flow of the information.

3) Gradient flow preservation:- Prevents gradients from

vanishing, allowing deep networks to learn effectively.

How LSTM networks solve the problem of vanishing gradients?

→ By using gated mechanisms that regulate the flow of information through time steps. In traditional RNNs, gradients shrink exponentially as they propagate backward, causing earlier layers to learn very little. LSTMs prevent this by maintaining long-term dependencies efficiently.

→ Process it uses —

1) Memory cell retention:- LSTM maintains cell state that stores long-term information. This information can flow unchanged through multiple time steps, preserving gradient strength.

2) Forget gate:- Controls whether past information should be discarded or retained. Prevents unnecessary buildup, reducing loss of important data.

3) Input gate:- Regulates how much new information is added to the memory cell. Helps prevent over-

loading the network with irrelevant details.

4) Output gate:- Ensures only relevant information is passed forward. Prevents unnecessary weight updates, stabilizing gradients.

4) Bidirectional capability:- Can process sequences in both directions, capturing more context.

• Advantages of LSTM:- 1) It handles sequential data better than traditional RNNs.

2) Avoids vanishing gradients issues.

3) Captures long-term dependencies efficiently.

4) Widely used in NLP & speech synthesis.

• Applications:- 1) Text generation & chatbots.

2) Machine Translation (Google Translate).

3) Stock Market Forecasting.

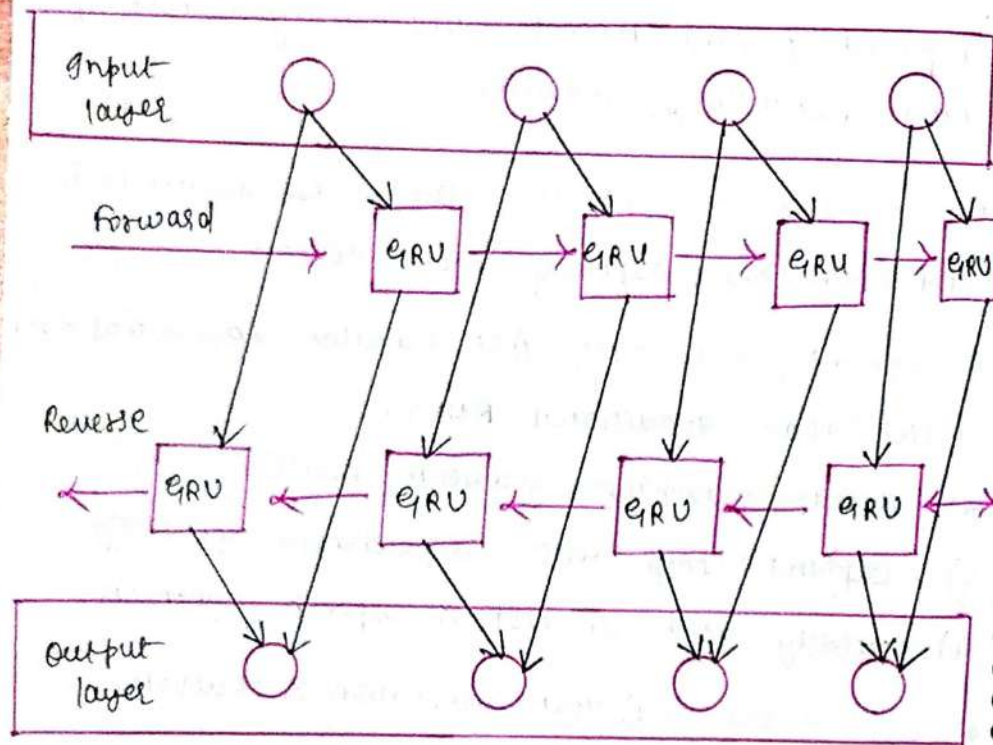
4) Speech recognition (Siri, etc).

5) Medical diagnosis Predictions.

Gated Recurrent units:-

• Gated recurrent units are a type of RNN architecture that is designed to handle the vanishing gradient problem in traditional RNNs. GRUs are

similar to LSTM networks but have fewer parameters, making them faster to train.



GRUs have two main components -

1) Update gate :- This gate determines how much of the previous hidden state to keep and how much to update with new information.

2) Reset gate :- This gate determines how much of the previous hidden state to forget.

GRUs are simpler and faster to train than long short-term memory networks, but they

can still learn complex patterns in data.

• Architecture :-

1) Input gate :- This gate determines how much of the new information to add to the hidden state.

2) Reset gate :- This gate determines how much of the previous hidden state to forget.

3) Hidden state :- This is the internal memory of the GRU, which captures information from previous time steps.

4) Output gate :- This gate determines how much of the hidden state to output.

• GRU vs LSTM :-

Feature	GRU	LSTM
Number of gates	2 Reset & update.	3 (Forget, In and Out).
Cell state	No separate cell state.	Distinct memory cell.
Complexity	Lower	Higher
Performance	Works well for short sequences	Better for long sequences.

• Applications :- Chatbots, Machine translation, etc.

Unit-5

Restricted Boltzman Machines :-

- Restricted Boltzmann machines are stochastic neural networks used for feature learning, dimensionality reduction, and unsupervised learning.

- They were popularized by Geoffrey Hinton and played a key role in early deep learning models like Deep Belief Networks (DBNs).

- Key features of RBMs —

1) Two-layer architecture :- RBMs consist of a visible layer and a hidden layer, with no connections within each layer.

2) Two-based model :- They minimize an energy function, ensuring optimal feature representation.

3) Binary neurons (stochastic) :- Neurons are binary (values 0 or 1) and are updated probabilistically.

4) Weight learning via CD :- Uses unsupervised learning to adjust weights efficiently.

- Architecture :-

1) Visible units (v) :- Represent input features.

2) Hidden units (h) :- Capture meaningful patterns.

3) The weight matrix (w) :- Connects visible and hidden layers.

4) Bias terms (b_v, b_h) :- Adjusts activation behavior.

- RBMs do not have intra-layer connections, neurons in the visible layer are NOT connected to each other.

neurons in the hidden layer are NOT connected to each other.

- Training process using Contrastive Divergence :-

- 1) Initialize random weights.
- 2) Perform forward pass.
- 3) Perform reconstruction.
- 4) Adjusts weights using gradient descent.

- Applications :- 1) Feature learning.

2) Dimensionality reduction.

3) Anomaly detection.

4) Recommendation system.

5) Generative models.

Gibbs sampling :-

- Gibbs sampling is a Markov chain Monte Carlo algorithm used to train restricted Boltzmann machines.

- RBMs are a type of generative model that consists of a visible layer and a hidden layer.

- Gibbs sampling works by iteratively sampling the hidden units given the visible units, and then sampling the visible units given the hidden units.

- This process creates a Markov chain that converges to the equilibrium distribution of the RBM.

- Gibbs sampling is useful for training RBMs because it allows us to —

1) approximate the partition function :- Gibbs sampling can be used to estimate the partition function of the RBM, which is necessary for computing the likelihood of the data.

2) sample from the model :- Gibbs sampling can be used to generate samples from the RBM, which can be used to evaluate the quality of the

model.

- RBMs are energy-based models that require sampling to learn data distributions. Gibbs sampling provides an efficient way to generate realistic samples from the RBM's probability distribution.

• Steps in Gibbs sampling -

- 1) Initialize visible units.
- 2) Sample hidden unit (h_t) from visible units (v_t).
- 3) Sample visible unit (v_{t+1}) from h_t .
- 4) Repeat multiple steps.
- 5) Contrastive divergence (CD-k)

- Advantages :- 1) approximate data distribution.

2) Efficient training.

3) Contrastive divergence.

4) Improves feature learning.

5) Essential for DBNs.

Deep belief networks :-

- DBNs are a class of deep learning models that consists of multiple layers of RBMs stacked together.

- They are designed for unsupervised pretraining followed by fine-tuning with supervised learning techniques.

- Developed by Geoffrey Hinton and his colleagues, DBNs have been instrumental in feature extraction, dimensionality reduction, and generative modelling.

- Key features of DBNs -

- 1) Layer-wise training:- DBNs are trained one layer at a time using unsupervised learning before fine-tuning the full network.

- 2) Stacked RBMs:- Each layer is a RBM that learns hierarchical features.

- 3) Probabilistic generative models:- DBNs estimate data distributions, making them useful for unsupervised learning.

- 4) Contrastive divergence for pretraining:- Instead of backpropagation, Gibbs sampling and CD are used for weight learning.

- Architecture:-

- 1) Input layer:- Represents raw data. (e.g. - images, text).

- 2) Stacked RBMs:- Each RBM extracts higher-level features.

- 3) Final layer:- Used for classification or regression.

- Training process:- Each RBM is trained using Gibbs sampling and CD. This improves feature representation without labelled data.

Once RBMs are stacked, the network can be fine-tune using backpropagation for classification tasks.

- Advantages:- 1) Better feature representation.

- 2) Reduced need for labelled data.

- 3) Handles complex data.

- 4) Improves training stability.

- 5) Used for speech recognition & image classification.

Markov Networks:-

- Markov network are a type of undirected probabilistic graphical model used for modelling complex relationships between variables in probability distribution. Unlike Bayesian networks, which use directed edges, Markov network rely on undirected

edges to express dependencies.

• Key features :-

1) Undirected graph representation :- Nodes represent random variables and edges capture dependencies between them.

2) Conditional independence :- Two variables are conditionally independent if there is no edge connection between them.

3) Factorization with cliques :- Probabilities are factorized using potential functions over cliques in the graph.

4) Energy-based model :- often used in image segmentation, speech recognition and NLP.

• Applications :-

- 1) Image processing.
- 2) Text mining and NLP.
- 3) Social network analysis.
- 4) Genomics & Bioinformatics.
- 5) Speech recognition systems.

Generative Adversarial Networks :-

Generative adversarial networks (GANs) are a type of deep learning model that consists of two neural networks :-

1) Generator :- generates new samples that resembles the training data.

2) Discriminator :- evaluates the generated samples and tells the generator whether they are realistic or not.

The generator and discriminator are trained simultaneously, with the generator trying to produce more realistic samples, and the discriminator trying to correctly distinguish between real and generated samples.

GANs have been used for :-

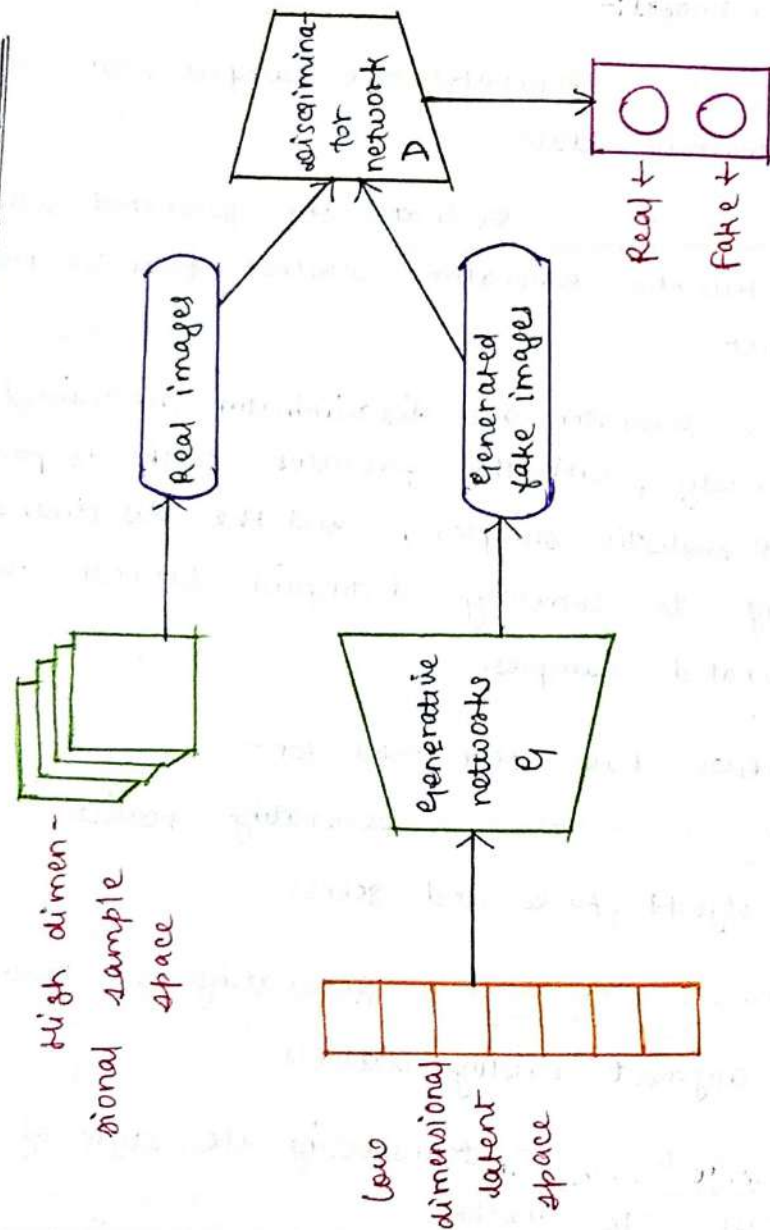
1) Image generation :- generating realistic images of objects, faces and scenes.

2) Data augmentation :- generating new training data to augment existing datasets.

3) Style transfer :- transferring the style of one image to another.

GENERATIVE ADVERSARIAL NETWORKS [GANs]:

- GANs have shown impressive results in generating realistic data, but they can be challenging to train and require careful tuning.



Training process:-

- 1) The generator receives input as random noise.
- 2) The generator produces synthetic data.
- 3) The discriminator predicts whether the data is real or fake.
- 4) Adversarial learning:- a) If the discriminator classifies correctly, the generator updates its weights to improve.
b) If the discriminator fails, it updates to improve its ability to distinguish fake data.
c) This cycle continues until the generator produces high-quality outputs.

Applications:-

- 1) Image generation.
- 2) Data augmentation for medical imaging.
- 3) Super-resolution & image restoration.
- 4) Video animation & synthesis.
- 5) Game development & texture generation.

Pixel RNN:- Pixel RNN is a deep learning architecture designed for image

generation using sequential modelling techniques.

- Unlike traditional convolutional network models that generate images all at once, Pixel RNN generates image pixel-by-pixel, processing each pixel sequentially. It was introduced by Google Deepmind in 2016.

- Key features:- 1) Sequential Pixel generation:-

Processes pixel one by one instead of generating an entire image at once. Ensures spatial dependencies are captured effectively.

2) uses RNNs:- Utilizes LSTMs / GRUs to model pixel relationships. Allows complex dependencies across pixel values.

3) Autoregressive model:- Each pixel depends on previously generated pixels. Captures local & global image structure.

4) Conditional image generation:- Can generate images conditioned on labels.

- Architecture:- 1) Row LSTM:- Uses recurrent layers to process rows of pixels sequentially. It

captures dependencies across horizontal pixels first.

2) Diagonal LSTM:- Extends Row LSTM by processing pixels diagonally, improving image consistency. Helps model spatial correlation more effectively.

3) Pixel CNN:- A faster variant using convolutional layers instead of RNNs. Improves efficiency while maintaining autoregressive properties.

- Applications:- 1) Image completion & Restoration.

2) Super-resolution.

3) Autoregressive image generation.

4) Handwriting synthesis.

5) Texture generation for games & design.